PROGRAMACION ORIENTADA A OBJETOS

Definiciones y Introducción a C#

Ing. Andrés Eduardo Paternina Ariza Esp.

Universidad del Magdalena

apaternina@unimagdalena.edu.co

Detalle del curso



24 sesiones



Tres examenes:

Sesión 8 – 100 puntos

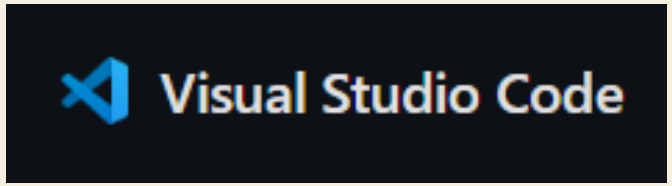
Sesión 17 – 100 puntos

Sesión 21 y 22 – 150 puntos

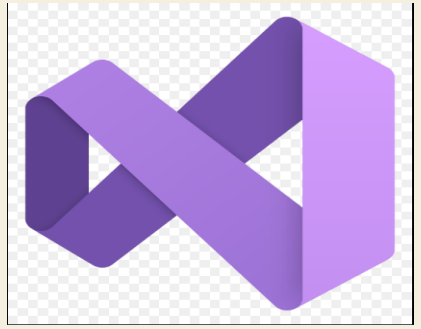


Actividades : 50

Herramientas a utilizar







Agenda de hoy

1. Definiciones: lenguaje interpretado y lenguaje compilado
2. Definiciones: paradigmas de programación
3. Introducción a C#
4. Ejercicios de calentamiento

¿Qué es un lenguaje interpretado?

La definición

- Un **intérprete** lee el código y lo ejecuta línea por línea, en el momento de correrlo.
- Necesitas el intérprete instalado en la máquina donde corre el programa.
- No hay paso previo de compilación: escribes y ejecutas de inmediato.
- Los errores de tipo o de nombre aparecen cuando esa línea llega a ejecutarse, no antes.

Ejemplos

- **Python**

¿Qué es un lenguaje compilado?

La definición

- Un **compilador** traduce todo el código antes de ejecutarlo y produce un archivo ejecutable.
- El ejecutable corre sin necesidad del código fuente y suele ser más rápido.
- Son dos pasos: primero compilas, después corres el programa.
- El compilador detecta errores de tipo y de nombre antes de que el programa llegue al usuario.

Ejemplos

- **C**
- **Java**

¿Qué es un paradigma de programación?

La definición

- Un paradigma de programación es una forma o enfoque general de organizar, estructurar y pensar la solución de un problema mediante código.
- Cambiar de paradigma cambia cómo piensas el problema, no la sintaxis.

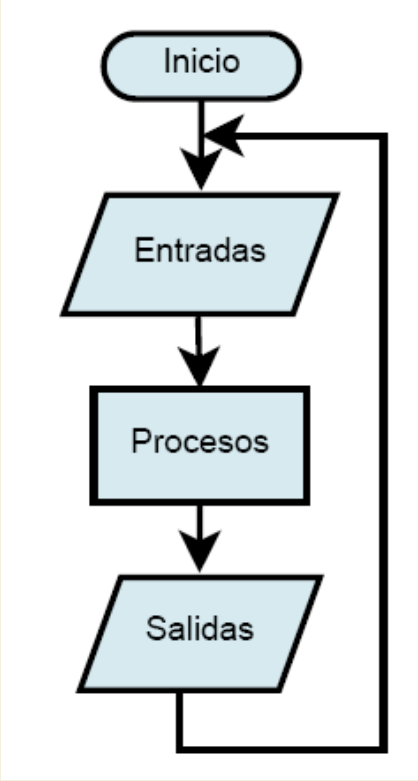
Tipos

- **Estructurada**
- **Orientada a objetos**
- **Funcional**
- **Lógica y declarativa**

Programación estructurada

La definición

- El programa es una **secuencia de pasos**: entrada, proceso, salida.
- Se organiza en **funciones** que reciben datos, los transforman y devuelven resultados.
- Los datos y las funciones que los usan viven separados.
- Funciona bien mientras el programa es pequeño y los pasos son claros.



Programación orientada a objetos

La definición

- El programa **modela cosas del mundo real**: una venta, un pedido, un producto.
- Cada **objeto** guarda sus datos y sabe hacer cosas con ellos.
- Los datos y el comportamiento viajan juntos, así que el programa crece sin desordenarse.



Problema - Tres cuentas de banco

Un banco necesita manejar tres cuentas. Cada cuenta tiene titular y saldo, y permite consignar y retirar dinero.

Datos
Titular y saldo de cada cuenta.

Operaciones
Consignar y retirar dinero.

Estructurada

```
titulares = ["Ana", "Luis", "Sara"]
saldos    = [100000, 50000, 20000]

def consignar(i, monto):
    saldos[i] += monto

def retirar(i, monto):
    if monto <= saldos[i]:
        saldos[i] -= monto
    else:
        print("Fondos insuficientes")

consignar(0, 50000)
retirar(1, 80000)
```

Orientada a objetos: cada cuenta se encarga de lo suyo

Cuenta de Ana

SUS DATOS
titular: Ana
saldo: $100.000

LO QUE SABE HACER
consignar dinero
retirar dinero

Cuenta de Luis

SUS DATOS
titular: Luis
saldo: $50.000

LO QUE SABE HACER
consignar dinero
retirar dinero

Cuenta de Sara

SUS DATOS
titular: Sara
saldo: $20.000

LO QUE SABE HACER
consignar dinero
retirar dinero

Limitaciones cuando el código crece

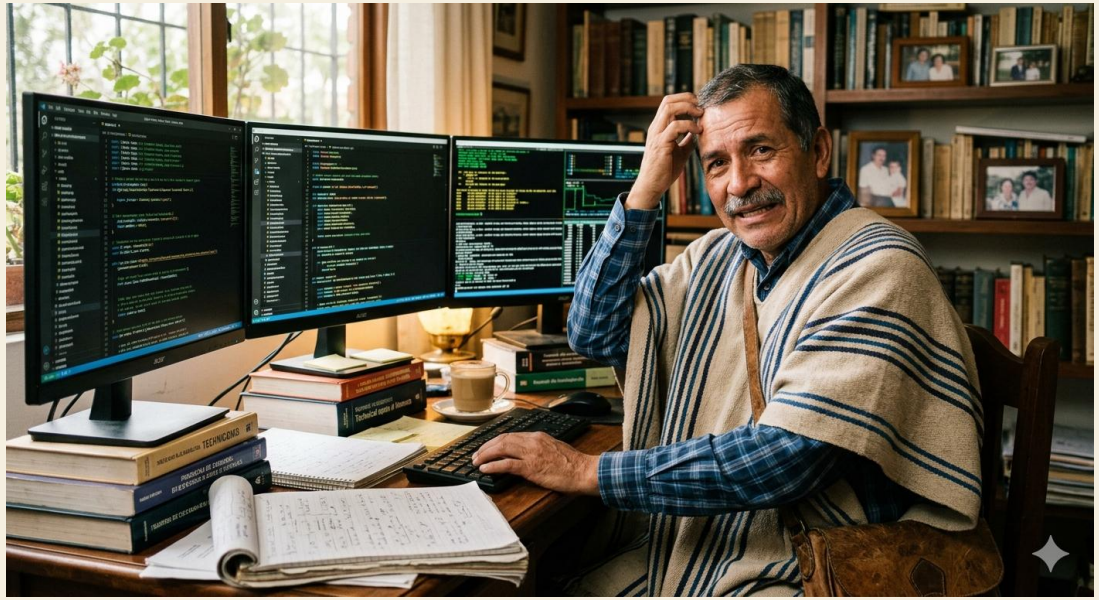
Variables globales que cualquiera modifica → bugs invisibles.

Funciones acopladas: cambiar una rompe otras tres.

Código duplicado para entidades parecidas (cliente, proveedor, empleado...).

Difícil de testear: la lógica está dispersa en procedimientos.

Onboarding lento: ningún archivo "explica" el dominio.



Ventajas de la POO



Encapsulamiento



Reutilización



Mantenibilidad



Modela el mundo real

Desventajas y cuándo no usarla

Tiene costos

- **Curva de aprendizaje**: clases, herencia y polimorfismo requieren tiempo para dominarse.
- **Más código inicial**: una clase puede ser overkill para un script de 20 líneas.
- **Sobre-diseño**: jerarquías muy profundas se vuelven difíciles de seguir.

Cuando no es lo mejor

- **Scripts pequeños y automatizaciones** simples donde crear muchas clases agrega complejidad innecesaria.
- **Procesamiento de datos** donde enfoques funcionales, pipelines o SQL resultan más naturales y expresivos.
- **Cómputo numérico o científico intensivo** donde el rendimiento y las operaciones vectorizadas son prioridad..

C#

¿Por qué C#?

1

Tipado fuerte

El compilador detecta errores antes de ejecutar. Refactorizar es seguro.

2

Lenguaje moderno

Sintaxis clara, propiedades, LINQ diseñado para POO real.

3

Industria amplia

Backend (.NET), juegos (Unity), apps Windows, móvil (MAUI), cloud (Azure).

Hola, mundo en Python y C#

PYTHON

```python
# hola_mundo.py

print("¡Hola, mundo!")
```

C#

```csharp
// Program.cs

Console.WriteLine("¡Hola, mundo!");
```

Variables y tipos en Python vs C#

PYTHON

```python
nombre = "Ana"
edad = 25
peso = 60.5
altura = 1.65
# ambos son float: un solo tipo
activo = True
```

C#

```csharp
string nombre = "Ana";
int edad = 25;
float peso = 60.5f;
double altura = 1.65;
bool activo = true;
decimal saldo = 100m;
```

Strings y convenciones de nombres

STRING INTERPOLATION

Python (f-string)

```
nombre = "libros"
precio = 25
print(f"{nombre}: ${precio}")
```

C# — sintaxis con $

```
string nombre = "libros";
int precio = 25;
Console.WriteLine(
  $"{nombre}: ${precio}");
```

CONVENCIONES DE NOMBRES

Clases y métodos

`calcular_total` → `CalcularTotal`

Variables locales

`precio_total` → `precioTotal`

Constantes

`IVA_RATE` → `IvaRate`

Archivos

`producto.py` → `Producto.cs`

Condicionales: if / else

PYTHON

```python
edad = 20

if edad >= 18:
  print("Mayor de edad")
elif edad >= 13:
  print("Adolescente")
else:
  print("Niño")
```

C#

```csharp
int edad = 20;

if (edad >= 18) {
  Console.WriteLine("Mayor");
} else if (edad >= 13) {
  Console.WriteLine("Adolescente");
} else {
  Console.WriteLine("Niño");
}
```

Ejercicio Mayor o menor de edad

Dada la edad de una persona, indica si es mayor de edad o menor de edad.

ENTRADA
La edad en años.

REGLA
18 años o más: mayor de edad.

SALIDA
Un mensaje con el resultado.

Mayor o menor de edad, resuelto en C#

C#

```
int edad = 19;
if (edad >= 18)
{
    Console.WriteLine("Mayor de edad");
}
else
{
    Console.WriteLine("Menor de edad");
}
```

Bucles: for, while y do-while

PYTHON

```python
for i in range(3):
    print(i)

i = 0
while i < 3:
    print(i)
    i += 1
```

C#

```csharp
for (int i = 0; i < 3; i++) {
    Console.WriteLine(i);
}

int j = 0;
while (j < 3) {
    Console.WriteLine(j);
    j++;
}

int k = 0;
do {
    Console.WriteLine(k);
    k++;
} while (k < 3);
```

Ejercicio — Tabla de multiplicar

Calentamiento con bucles

Dado un número, mostrar su tabla de multiplicar del 1 al 10.

ENTRADA
Un número entero.

PROCESO
Multiplicarlo por 1, 2, 3… hasta 10.

SALIDA
Diez líneas: 7 x 3 = 21.

PISTA Diez repeticiones conocidas: es el caso del for, no del while.

Tabla de multiplicar, resuelta con for en C#

C#

```
int n = 4
for (int i = 1; i <= 10; i++)
{
    Console.WriteLine(
        $"{n} x {i} = {n * i}"
    );
}
```

Funciones / Métodos

Python

```python
def saludar(nombre):
    return f"Hola, {nombre}"

mensaje = saludar("Ana")
print(mensaje)

def imprimir(nombre):
    print(f"Hola, {nombre}")

imprimir("Luis")
```

C#

```csharp
string Saludar(string nombre) {
    return $"Hola, {nombre}";
}
string mensaje = Saludar("Ana");
Console.WriteLine(
        $"{mensaje}");


void Imprimir(string nombre) {
    Console.WriteLine(
        $"Hola, {nombre}");
}
Imprimir("Luis");
```

Ejercicios — Funciones

1 · SIN RETORNO

Crear una función que reciba dos números y imprima la suma de ambos.

Salida esperada

```
La suma es 12
```

2 · CON RETORNO

Crear una función que reciba dos números y retorne la suma de ambos. Mostrar el resultado en el programa principal.

Salida esperada

```
La suma es 12
```

Las dos funciones, resueltas en C#

void para mostrar en pantalla, int para devolver un valor

1 · SIN RETORNO

```
void Sumar(int a, int b)
{
    Console.WriteLine(
        $“La suma es, {a + b}");
}

Sumar(5, 7);
```

2 · CON RETORNO

```
int Sumar(int a, int b)
{
    return a + b;
}

int total = Sumar(5, 7);
Console.WriteLine(
    $"La suma es {total}");
```

Tarea

Crea un programa en C# que, con las notas de 3 parciales, calcule el promedio y muestre:
“Aprobado” si el promedio es mayor o igual a 3.0.
“Recuperación” si está entre 2.0 y 2.9.
“Reprobado” si es menor a 2.0.
Además, debe indicar cuál de las tres notas fue la más alta.